Tokenizer or Training Data? How a Small Uzbek Language Model Beats Much Larger Multilingual Models

Islombek Turdiyev

Abstract

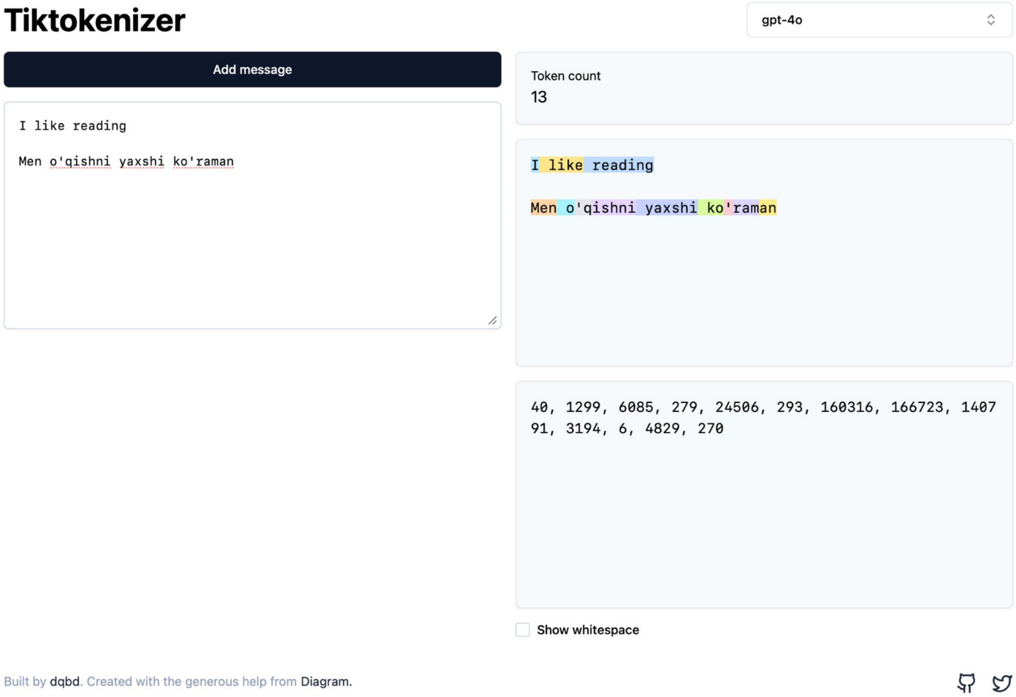
Large language models split Uzbek text into far more tokens than English text, which makes them slower and more expensive for Uzbek speakers. *Objectives:* I asked what matters more for an efficient Uzbek language model: a tokenizer built for the language, or training the model from scratch on Uzbek text. *Methodology:* I built a 16,384-token byte-level BPE tokenizer for Uzbek and used it to train a 103M-parameter decoder from scratch on about 1.06 billion Uzbek tokens. I compared it with mGPT-1.3B, both zero-shot and fine-tuned with QLoRA; with four current Qwen models of 0.5 to 1.7 billion parameters; and with an ablation model trained on exactly the same documents but with mGPT's tokenizer. All models were scored by bits-per-byte, a measure of how well a model predicts text, on 474 held-out documents in their original web spelling, none of which share a passage with the training data. *Findings:* My tokenizer used the fewest tokens of the eleven tested (1.749 per word): about half as many as GPT-2 and 21% fewer than XLM-R. My model predicted held-out Uzbek text better than every multilingual model. It needed about 4% fewer bits than the fine-tuned mGPT (1.038 against 1.077 bits-per-byte; 95% CI for the difference [0.031, 0.049]) and 32% fewer than the strongest Qwen model, which is 17 times larger. The ablation model reached comparable quality: 0.007 bits-per-byte better than mine on web spelling and 0.005 worse on normalized spelling. So the advantage over mGPT comes from pretraining on Uzbek text, not from the tokenizer. What the tokenizer bought was efficiency: comparable quality with less than half the parameters and about a third of the training computation. Fine-tuning mGPT on ten times more data narrowed the gap but did not close it. An audit of my own work found four problems, each of which made my results look better than they were; I corrected all four, and re-scoring my original test set with the corrected code comes within 0.003 bits-per-byte of my original numbers, so each correction can be checked. *Contributions:* to my knowledge, the first controlled test for Uzbek of whether a dedicated model's advantage over a fine-tuned multilingual model comes from its tokenizer or from its training data, which shows that the tokenizer's benefit is efficiency rather than prediction quality; a dedicated Uzbek tokenizer and model; a documented audit; and openly released tokenizer, model weights, code, and evaluation data.

Keywords: large language models, tokenization, fine-tuning, low-resource languages, Uzbek.

1. Introduction

Large language models (LLMs) are trained mostly on English, and they work far better in English than in the thousands of other languages people speak, especially low-resource ones (Qin et al., 2025). Uzbek, a Turkic language of Central Asia with roughly 40 million speakers, is one of the languages left behind (Bobojonova et al., 2025). The gap is not only about quality. Commercial LLMs are priced and limited by how text is processed, so speakers of underserved languages often pay more, wait longer, and get weaker answers than English speakers for the same task (Petrov et al., 2023). As LLMs become everyday tools for education, work, and information, this is a question of fairness and access, not only of engineering.

For Uzbek, much of the disadvantage starts at one early step: tokenization. A tokenizer converts text into the numbers a model reads, and converts the model's output back into text. Figure 1 shows the problem. An English-centric tokenizer splits "I like reading" into 3 tokens, but splits its Uzbek translation into 9, breaking single Uzbek words into several fragments.



```
I like reading

Men o'qishni yaxshi ko'raman
```

Token count
13

```
I like reading

Men o'qishni yaxshi ko'raman
```

```
40, 1299, 6085, 279, 24506, 293, 160316, 166723, 1407
91, 3194, 6, 4829, 270
```

☐ Show whitespace

Figure 1. The same sentence tokenized by an English-centric tokenizer (GPT-4o's o200k_base). The English sentence "I like reading" becomes 3 tokens, while its Uzbek equivalent "Men o'qishni yaxshi ko'raman" becomes 9, with single words broken into several pieces; the one remaining token of the 13 shown is the blank line between the sentences. Screenshot taken by the author from the Tiktokenizer web tool (tiktokenizer.vercel.app).

In this study I built a dedicated Uzbek tokenizer and a decoder model trained from scratch, and compared them with multilingual models, including one fine-tuned on a small part of the same Uzbek corpus. My goal was twofold: to find out what drives efficient language modeling for a low-resource language like Uzbek, a dedicated tokenizer or a model trained from scratch, and to weigh the cost of each approach against its benefit. I examined this through three research questions:

1. Which tokenizer is more efficient for a low-resource language: one built for it, or an existing multilingual one?
2. Which model predicts held-out text in that language better: a monolingual model built from scratch, or a multilingual model fine-tuned on the language?
3. When is it worth building from scratch rather than adapting an existing model?

2. Literature Review

2.1 Tokenization inequity

A clear problem identified in the literature is that current LLMs treat low-resource languages unfairly, and that much of this starts at tokenization, before the model even runs (Petrov et al., 2023). Measuring the same text across many languages, Petrov et al. (2023) found that some languages are split into up to 15 times more tokens than English, and that the gap appears in all 17 tokenizers they tested, including ones built for multilingual use such as mBERT, XLM-R, and BLOOM. The disadvantage is therefore systematic, not the fault of one bad tokenizer.

This token gap causes three practical harms (Petrov et al., 2023). The first is cost: providers charge per token, so users of some languages pay at least 2.5 times more than English users for the same content. The second is speed: more tokens mean more computation, so responses are slower. The third is context: a model can hold only a fixed number of tokens at once, so a language that needs more tokens fits less actual content into the model's working memory. A study of OpenAI's commercial API confirmed these harms in practice: users of many languages paid more for worse results, and these languages were often spoken where the API was already least affordable (Ahia et al., 2023).

Recent work tries to reduce these harms in two main ways. The first patches a trained model. Churchill and Skiena (2026) add new tokens to several popular models after training, merging frequent multi-token sequences into single tokens and lowering the token count without retraining. The second redesigns the tokenizer. MAGNET (Ahia et al., 2024) adjusts how aggressively text is split so that low-resource languages are not broken into tiny pieces, and reports more balanced segmentation across languages and about a threefold reduction in token length for Indic languages. Both approaches start from an existing multilingual model and try to repair it. I ask a different question: whether building a tokenizer from scratch for a single low-resource language is a more direct route to the same goal.

2.2 From-scratch models for low-resource languages

There are two ways to serve a low-resource language better: adapt an existing multilingual model by training it further on the language, or build a new model for that language from the start. A growing body of work finds the second route surprisingly competitive. Small models built from scratch with their own tokenizers have matched or beaten much larger multilingual models on their target language, at a small fraction of the training cost, especially for languages with limited data but complex word structure.

This has been shown for languages close to Uzbek. For Kazakh, a fellow Turkic language, the SozKZ models were trained from scratch with a Kazakh tokenizer and performed strongly against larger multilingual baselines (Tukenov, 2026). For Indic languages, the small Paramanu models reached competitive generation quality while being far smaller and cheaper to train than the multilingual models they were compared with (Niyogi & Bhattacharya, 2024). At a much larger scale, the Goldfish project trained small monolingual models from scratch, each with its own tokenizer, for 350 languages including Uzbek, and found that they often predict text in their language better than large multilingual models (Chang et al., 2024).

One point in this line of work is easy to miss. These studies usually compare their from-scratch model with a multilingual model used as it is, without any extra training on the target language. That can overstate the advantage of building from scratch, because a large multilingual model fine-tuned on the language might close the gap. A fair comparison should therefore include a fine-tuned version of the multilingual model, and this study does so.

2.3 Uzbek NLP

Natural language processing for Uzbek is active, but it consists almost entirely of encoders: models that read and analyze text rather than generate it. Several monolingual Uzbek BERT-style encoders exist, including UzBERT (Mansurov & Mansurov, 2021) and BERTbek (Kuriyozov et al., 2024), along with tools such as the BBPOS part-of-speech tagger (Bobojonova et al., 2025). These Uzbek encoders consistently beat the multilingual encoder mBERT on Uzbek tasks such as classification and tagging (Kuriyozov et al., 2024; Bobojonova et al., 2025), which is itself evidence that a model built for one language can beat a larger multilingual one.

Generative (decoder) models for Uzbek, the kind that write new text, are much rarer. GPTuz (Adilova et al., 2022) fine-tunes the English GPT-2 small on Uzbek news text, and the Goldfish suite includes small Uzbek models trained from scratch as part of its 350-language release (Chang et al., 2024). Multilingual decoders such as mGPT also generate Uzbek, but Uzbek is a small part of their training. Two gaps remain. First, to my knowledge no study has tested, for Uzbek, whether a dedicated model’s advantage over a multilingual one comes from its tokenizer or from its training data, or compared a dedicated model with a multilingual model fine-tuned on Uzbek. Second, even strong current systems still make errors with Uzbek’s agglutinative morphology, in which one root can carry many stacked suffixes; when a tokenizer splits such words poorly, models produce errors such as wrong suffixes and broken vowel harmony (Solidjonov & Najmiddinov, 2026). Filling the first gap, with a controlled experiment that holds the training documents fixed and changes only the tokenizer, is the main novelty of this work.

3. Methodology

3.1 Data and tokenizer

I trained on the Uzbek Latin-script part (uzn_Latn) of FineWeb-2, a large multilingual web corpus that has been cleaned, language-identified, quality-filtered, and deduplicated (Penedo et al., 2025). It contains 1.23 million documents, which my tokenizer encodes into 1.06 billion tokens: 955 million for training and 106 million for validation. I did no extra cleaning.

The tokenizer is a byte-level BPE tokenizer (byte-pair encoding; Sennrich et al., 2016) with a vocabulary of 16,384 tokens. Byte-level means it works on raw bytes, so it can encode any text, including other scripts, without failing. Before BPE, text is split with the GPT-2 regular expression (Radford et al., 2019). The tokenizer also normalizes apostrophes, which Uzbek uses inside letters (oʻ, gʻ) and inside words (maʼlumot), and which people type in several different ways. Its rules turn the right quote (’) and the backtick (`) into a plain apostrophe ('), turn o', O', g' and G' into the correct forms oʻ, Oʻ, gʻ and Gʻ, and turn an apostrophe between two letters into the tutuq belgisi (ʼ). Without this, one word typed four ways would be learned as four different token sequences.

3.2 Models

The from-scratch model (uzbek-gpt-103m). A decoder-only transformer with 103 million parameters: 12 layers, 768-dimensional embeddings, 12 attention heads, and a 1,024-token context. It uses components that are standard in current LLMs: RMSNorm (Zhang & Sennrich, 2019), rotary position embeddings or RoPE (Su et al., 2024), SwiGLU (Shazeer, 2020), and multi-head attention (Vaswani et al., 2017). One part differs from the standard design: after the attention heads are combined, there is no output projection, so each attention layer has about 590,000 fewer parameters than usual (7 million in total). I used an otherwise standard shape on purpose, because architecture is not the question this study asks. I trained it with AdamW (Loshchilov & Hutter, 2019; $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1) for about 9,700 steps, which is about two passes over the data. The learning rate rose linearly to 3×10^{-4} over the first 400 steps and then followed a cosine curve down to 3×10^{-5}. Each step used 192 sequences of 1,024 tokens (about 197,000 tokens), with bfloat16 mixed precision and gradient clipping at 1.0. Training took about 3.4 hours on one rented RTX 4090 GPU and cost about $3.60.

The ablation model. To separate the effect of the tokenizer from the effect of training on Uzbek text, I trained a second model with the same architecture and the same training settings on exactly the same training documents, but using mGPT's tokenizer instead of mine. mGPT's tokenizer has 100,000 tokens, which enlarges the embedding tables and makes this model 232 million parameters, more than twice the size of mine. It also cuts the same text into more tokens, so two passes over the same documents took 16,822 steps instead of about 9,700. Using the standard estimate of training computation (Kaplan et al., 2020), the ablation used about 2.8 times as much computation as my model. The comparison therefore holds the training data fixed, not the computation, and that difference favors the ablation model. I trained it for free on Kaggle, on two NVIDIA T4 GPUs over several sessions (about 51 hours at the logged 11 seconds per step). T4 GPUs do not support bfloat16, so it used float16 with loss scaling; the batch size, learning-rate schedule, and optimizer settings were the same as for my model. The first version of this model had two errors, described in Section 3.4; all results in this paper use the corrected retraining.

The baseline (mGPT-1.3B). mGPT is a multilingual GPT model with 1.3 billion parameters whose training data includes Uzbek (Shliazhko et al., 2024). I used it both as it is (zero-shot) and fine-tuned with QLoRA (Dettmers et al., 2023), which freezes the model in 4-bit precision and trains small low-rank adapters (LoRA; Hu et al., 2022) on top. I used rank 16 and alpha 32 on all linear layers, a 512-token training context, and 1 million tokens of the same Uzbek corpus, which updates 12.6 million of mGPT's 1.3 billion parameters. This fine-tuning takes 16 to 20 minutes on a free Kaggle T4 GPU.

Current multilingual models. mGPT was released in 2022. To check whether newer models handle Uzbek better, I also scored four current open base models without any fine-tuning: Qwen2.5 with 0.5 and 1.5 billion parameters (Qwen Team, 2024) and Qwen3 with 0.6 and 1.7 billion parameters (Qwen Team, 2025). They were trained on many trillions of tokens of multilingual text.

3.3 Evaluation

Metrics. For tokenizers I measured fertility (tokens per word, lower is better) and bytes per token. For models I measured bits-per-byte: how many bits a model needs, on average, to encode each byte of text (lower is better). I did not use perplexity, because perplexity is measured per token, and two models with different tokenizers cut the same text into different numbers of tokens, so their per-token scores are not comparable. Bits-per-byte divides by the raw bytes of text instead, which are the same for every model (Gao et al., 2020). I computed it over the whole test set at once: total loss divided by total bytes.

Held-out set. The test set is 474 documents from the validation split, about 200,000 words, none of which share any 24-token passage with the training data. Section 3.4 explains how it was built and why.

Scoring. I cut the test text into spans of about 800 bytes and scored each span with the 300 bytes before it as context (context that does not count toward the score). Every model scores the same spans, in the same text, with the same context. This matters because my tokenizer fits more text into each token: if each model were scored on chunks of 512 of its own tokens, as in my original design, my model would see about 2,400 bytes per chunk and mGPT only about 1,300, which gives my model extra context. The original design gave larger gaps in my favor; I report only the fairer one. I also capped every model's input at 512 tokens, the context length used to fine-tune mGPT. For the models in Table 2 and Table 4 it shortened the context of only 2 of the 2,058 spans.

Spelling. All headline results use the documents in their original web spelling, with every apostrophe exactly as it was typed. Each model applies its own tokenizer, including mine with its normalization. I also report results on the same documents in my tokenizer's normalized spelling, for comparison (Section 3.4 explains why this matters).

Statistics. Because every model is scored on the same text, I could use a paired bootstrap: resample the test set with replacement 10,000 times, recompute each difference between models, and take the middle 95% of the results as a confidence interval. I resample whole documents, not spans: each bootstrap sample draws 474 documents with replacement and keeps all of their spans, because spans from the same document are not independent of each other. (A span that crosses a document boundary is counted with the document that contains its middle.) My earlier versions resampled the 2,058 spans one by one, which gave intervals up to about three times narrower; every difference reported here excludes zero either way. If the interval does not include zero, the difference is not due to which documents happened to be in the test set.

3.4 Checking my own work

While revising this paper, I tested whether my evaluation and my control experiment were really what I thought they were. I found four problems. Each one made my results look better than they are, and I corrected all four before producing the results in Section 4.

1. The first test set was part of the training data. I had built my original test set by taking the first 200,000 words of the corpus. But the corpus had been split into training and validation data by position, with the first 90% used for training, so my test set was the opening of my training data. I confirmed this by taking 12 passages of 32 tokens from the test set and searching for them in the training file: 11 of the 12 were there, at matching positions. My model had been tested on text it had trained on twice. The fine-tuning data for the mGPT baseline had also overlapped this test set, which is why fine-tuning had appeared to help mGPT much more than it really does.

2. The validation data contained duplicates of the training data. Drawing the test set from the validation split instead did not fully solve the problem: 6 of 16 sampled passages still appeared in the training data, at scattered positions, because the web corpus itself contains duplicate pages. I therefore filtered the validation documents one by one. I recorded a fingerprint for every 24-token window of the training data (27.4 million fingerprints) and kept only validation documents with no fingerprint in common. Of 780 candidates, 306 (39.2%) were rejected. An independent check, an exact search for 16 passages from the final test set, found none of them in the training data.

3. The test text was written in my tokenizer’s spelling. The validation file stores text after my tokenizer’s normalization, so the test documents came out with every apostrophe converted to my tokenizer’s preferred forms. On the real web, people write these marks differently: in the original versions of the 474 test documents, the plain apostrophe (') makes up 70% of the marks in oʻ and gʻ, the left quote (‘) 19%, the right quote (’) 6%, and the official character (ʻ) only 5%. The other models learned from web text as it is typed, so the normalized spelling gave my model a home advantage. I recovered the original web version of all 474 documents from FineWeb-2, confirming each one exactly: applying my tokenizer’s normalization to the recovered document reproduces the test document character for character. The web-spelled version is the one used for all headline results.

4. The ablation model was not trained the way I had described it. When I compared the ablation’s training log with my description of it, I found two errors. Its data limit was meant to match my 955 million training tokens, but it counted words instead; the whole corpus has only about 574 million words, so it read every document, including the validation documents that the test set comes from. And it had stopped at step 10,000 of a planned 18,693, after about one pass over the data and before its learning rate had decayed. The second error held it back more than the first helped it: it scored 1.048 bits-per-byte on web text, 0.010 worse than my model, which made the tokenizer look more important than it is. I retrained it from the start, for two full passes, on exactly the documents my model trained on. I checked this in two ways: tokenizing those documents with my tokenizer reproduces the exact length of my training file, and none of 16 passages from the test set appears in the ablation’s new training data. The corrected ablation scores 1.031, slightly better than my model (Section 4.3).

To make the evaluation corrections checkable, I re-scored the original test set with the corrected evaluation code. It gives 1.1078, 1.1653 and 1.1604 bits-per-byte, within 0.003 of my originally reported 1.1050, 1.1628 and 1.1579. The small difference is in the last step: my original code divided by the byte count of the whole test file (1,697,029 bytes), including a final piece it never scored, instead of by the bytes it actually scored. Dividing my original per-chunk scores, which I kept, by the whole-file count gives back 1.1050 and 1.1627. I also checked whether my model had memorized the old test set. Scored with the same method on both test sets, my model did better on the new one by 0.081 bits-per-byte, but mGPT, which trained on neither, also did better by 0.056, so most of the difference is that the new text is easier. The remaining difference goes in the opposite direction from memorization, and is explained by the old set containing more Cyrillic text (0.51% of characters against 0.20%), which my Latin-script tokenizer handles poorly. I found no measurable memorization.

4. Results and Discussion

4.1 Tokenizer efficiency (RQ1)

Table 1. Tokenizer efficiency on the 474 held-out documents (200,818 words, original web spelling; sorted by fertility, lower is better). Created by the author.

Tokenizer	Vocab size	Fertility (tokens/word) ↓	Bytes/token ↑
uzbek-bpe-16k (this study)	16,384	**1.749**	**4.641**
XLM-RoBERTa	250,002	2.218	3.659
XGLM	256,008	2.497	3.250
mT5	250,100	2.502	3.244
GPT-4o (o200k_base)	200,019	2.578	3.148
mBERT	119,547	2.825	2.873
BLOOM	250,680	2.934	2.767
mGPT	100,000	3.159	2.570
GPT-3.5/4 (cl100k_base)	100,277	3.187	2.547
Qwen2.5	151,665	3.199	2.537
GPT-2	50,257	3.426	2.369

Note. Measured on the same documents as every model result in this paper, which my tokenizer never saw in training. An earlier measurement on a different sample of about 600,000 words gave 1.839 for my tokenizer, with the same ranking.

My tokenizer has the lowest fertility of the eleven, at 1.749 tokens per word. It beats XLM-R, whose vocabulary is about 15 times larger, and the tokenizers with vocabularies of around 250,000 range from 2.22 to 2.93 among themselves. Vocabulary size does not decide efficiency; being built for the language does. English-centric tokenizers do worst, and newer is not automatically better: GPT-4o's tokenizer is in the middle at 2.578, and Qwen2.5's, from a 2024 model, is second to last.

In practical terms, my tokenizer uses about half as many tokens as GPT-2 (3.426) and GPT-3.5/4 (3.187) for the same Uzbek text, 45% fewer than mGPT's tokenizer, and 21% fewer than XLM-R. Half the tokens means roughly half the cost under per-token pricing, faster generation, and about twice as much text fitting into the same context window. Prior work recovers some of this efficiency by patching existing models (Section 2.1); a tokenizer built for the language gets it directly.

4.2 Model quality (RQ2)

Table 2. Model comparison on the 474 held-out documents (bits-per-byte, lower is better; paired bootstrap over documents, 10,000 resamples). Created by the author.

Model	Size / Uzbek data	Web	Gap to mine [95% CI]	Normalized (gap)
uzbek-gpt-103m (from scratch)	103M / 1.06B	**1.0380**	—	**1.0281**
mGPT-1.3B + QLoRA	1.3B / 1M	1.0772	+0.0392 [0.0306, 0.0486]	1.0767 (+0.0486)
mGPT-1.3B zero-shot	1.3B / 0*	1.0801	+0.0421 [0.0336, 0.0514]	1.0774 (+0.0493)

Note. “Web” is the headline: the documents exactly as they appear on the web, in their original spelling. “Normalized” is the same documents in my tokenizer’s spelling (Section 3.4); the 95% confidence intervals for its gaps are [0.0398, 0.0585] and [0.0405, 0.0591]. A positive gap means my model is better. The ablation model is in Table 4. *Zero Uzbek training tokens in this study; mGPT’s multilingual pretraining includes an unknown, probably small, amount of Uzbek.

My model predicts held-out Uzbek text better than every multilingual model. On web-spelled text it reaches 1.038 bits-per-byte against 1.077 for mGPT fine-tuned with QLoRA and 1.080 for mGPT zero-shot. The difference from the fine-tuned model, 0.039 bits-per-byte, has a 95% confidence interval of [0.031, 0.049], well away from zero. In other words, my model needs about 4% fewer bits to encode the same text, at about one thirteenth of mGPT’s size.

Fine-tuning on 1 million tokens helped mGPT very little: 0.003 bits-per-byte on web text and less than 0.001 on normalized text. On my original, contaminated test set, the same fine-tuning had appeared to help by 0.041, because its training data overlapped that test set.

The two spellings also show a real difference between the models. Measured in bits per word, which can be compared across the two spellings because the words are identical, my model scores 8.428 on both. Its tokenizer normalizes apostrophes, so it does not matter how they were typed. The models trained on raw web text, including the ablation in Section 4.3, score about 1% better on web spelling than on normalized spelling (mGPT fine-tuned: 8.746 against 8.826), because web spelling is what they learned from. This is why all gaps are larger on the normalized text, and why I use web spelling as the fair comparison.

Current multilingual models do much worse on Uzbek than mGPT (Table 3). The best of the four Qwen models, Qwen3 with 1.7 billion parameters, needs 1.532 bits-per-byte: my model needs 32% fewer bits, at one seventeenth of the size. The pattern within the Qwen models is consistent: larger models beat smaller ones, and Qwen3, trained on more multilingual data, beats Qwen2.5 at the same size. mGPT, whose training data explicitly includes Uzbek, is by far the strongest of the multilingual models. This matches what Section 4.3 finds: for prediction quality, how much Uzbek text a model has seen matters more than its size or how recent it is. These models were not fine-tuned on Uzbek, and some of the test pages may have been in their training data, which cannot be checked; both points would favor them, not my model.

Table 3. Current multilingual models, zero-shot (web spelling, the same spans as Table 2; paired bootstrap over documents, 10,000 resamples). Created by the author.

Model	Parameters	Bits-per-byte ↓	Gap to mine [95% CI]
uzbek-gpt-103m (mine)	0.10B	**1.038**	—
Qwen3-1.7B-Base	1.72B	1.532	+0.493 [0.470, 0.516]
Qwen3-0.6B-Base	0.60B	1.905	+0.867 [0.840, 0.894]
Qwen2.5-1.5B	1.54B	1.948	+0.909 [0.883, 0.935]
Qwen2.5-0.5B	0.49B	2.357	+1.319 [1.289, 1.347]

The picture looks different if models are compared per token. There, mGPT seems much better: about 1.97 nats per token against 3.33 for my model, measured with my original chunking on normalized text. But this comparison is not fair. mGPT's tokenizer cuts text into more, smaller pieces, and small pieces are individually easier to predict, so mGPT earns a lower per-token loss by doing less work per token. That is why per-token loss cannot be compared across tokenizers, and why bits-per-byte is the headline metric. My result agrees with work on Kazakh (Tukenov, 2026) and Indic languages (Niyogi & Bhattacharya, 2024) finding that small dedicated models can compete with much larger ones, although those studies measure task accuracy rather than bits-per-byte.

4.3 What the advantage is made of

Table 2 compares two complete approaches, and they differ in several ways at once: tokenizer, size, and how much Uzbek text each model saw. Two experiments separate these.

The tokenizer. The ablation model was trained on exactly the same documents as mine, with mGPT's tokenizer instead of my own (Table 4).

Table 4. Tokenizer ablation: two models trained on the same documents. Created by the author.

	uzbek-gpt-103m	Ablation
Tokenizer (vocabulary)	uzbek-bpe-16k (16,384)	mGPT (100,000)
Parameters	103M	232M
Training steps (two passes)	about 9,700	16,822
Training computation	1×	about 2.8×
Bits-per-byte, web spelling	1.0380	**1.0310**
Bits-per-byte, normalized spelling	**1.0281**	1.0330
Bits per word, web spelling	8.428	**8.371**
Bits per word, normalized spelling	**8.428**	8.468

Note. Lower is better; bold marks the better model on each row. Difference, ablation minus mine, with 95% CI (paired bootstrap over documents, 10,000 resamples): web spelling −0.0070 [−0.0086, −0.0054]; normalized spelling +0.0049 [+0.0024, +0.0076]. Training computation is estimated from parameters and tokens (Kaplan et al., 2020).

The two models reach comparable quality. On web spelling the ablation is better by 0.007 bits-per-byte; on normalized spelling mine is better by 0.005. Both differences are real (neither interval includes zero), but they are small, they point in opposite directions, and each is less than a fifth of the 0.039 gap between my model and fine-tuned mGPT.

The winner probably changes because of what each model trained on. My tokenizer normalizes apostrophes, so my model learned only the normalized spelling, and it scores the same on both versions of the text (8.428 bits per word). mGPT's tokenizer has no normalization, so the ablation learned the web spelling, and like the other web-trained models it does about 1% better on web text (8.371 against 8.468 bits per word). On each version of the text, the model that trained on that spelling wins. I did not test this explanation directly. One more effect favors my model on web text: because my tokenizer turns every typed form of the apostrophe into the same character, my model is never asked to predict which form a writer used, while the ablation is. I did not measure how large this effect is, but it means the ablation's lead on web text is, if anything, understated.

So the tokenizer does not explain my model's advantage over mGPT. The ablation uses mGPT's own tokenizer, and it is 0.046 bits-per-byte ahead of the fine-tuned mGPT on web text, even further ahead than my model is. The advantage comes from pretraining on a billion tokens of Uzbek text. My first version of the ablation had suggested that the tokenizer was responsible for about a quarter of it; that result came from the flawed run described in Section 3.4.

What the tokenizer does change is cost. The ablation needed 232 million parameters, 16,822 training steps and about 2.8 times my model's training computation to reach comparable quality. After training, my model also does less work for every text it reads or writes, because it uses about 45% fewer tokens and each token is cheaper to process. This comparison held the training data fixed, not the computation: I did not train my model for as long as the ablation, so I cannot say how the two would compare at equal computation.

More fine-tuning data. A fair question is whether fine-tuning mGPT on more Uzbek text would close the gap. I fine-tuned it on 1 million and 10 million tokens, taken from training data that does not overlap the test set, and scored each version (Figure 2).

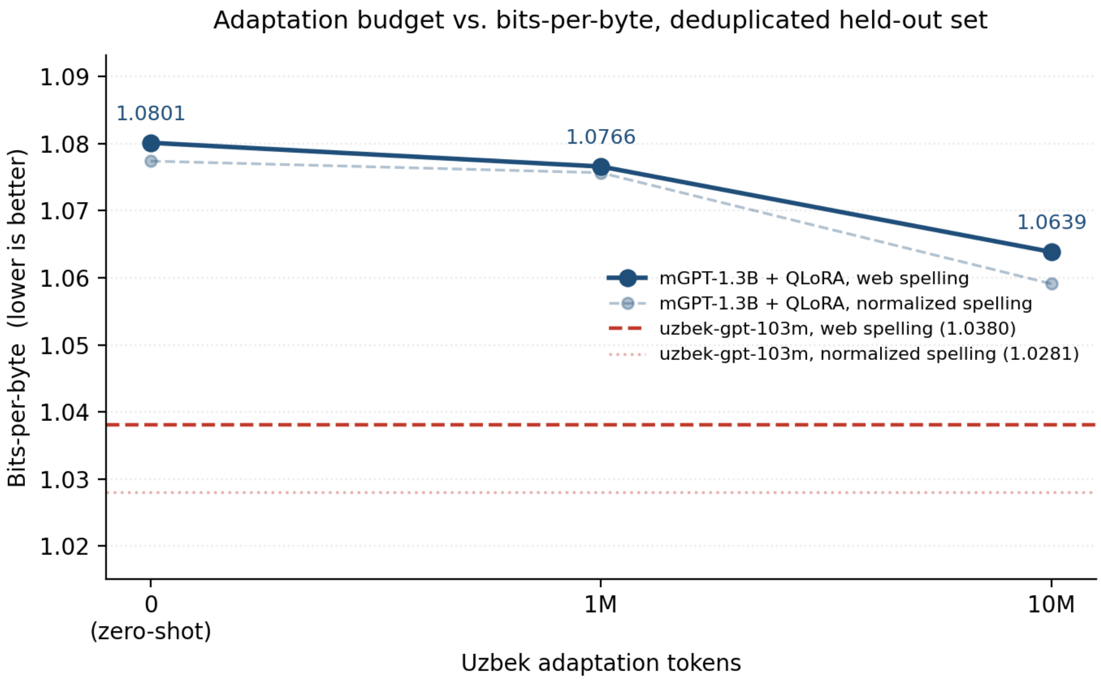


Figure 2. Bits-per-byte of mGPT-1.3B fine-tuned with QLoRA on 0, 1 million, and 10 million Uzbek tokens. Solid blue: web spelling (the headline). Faded blue: my tokenizer's normalized spelling. Red lines: my model, on each spelling. The 1M point (1.0766) comes from a separate fine-tuning run from the one in Table 2 (1.0772), whose training examples were drawn and shuffled slightly differently from the same opening documents of the corpus; QLoRA training is also not exactly repeatable from run to run. Created by the author.

Fine-tuning does keep helping. Going from 1 million to 10 million tokens improved mGPT by 0.013 bits-per-byte on web text, so it does not stop improving after a small amount of data. But at 10 million tokens it is still 0.026 bits-per-byte behind my model (95% CI [0.018, 0.034]). A rough extrapolation from these two points suggests that, if it kept improving at the same rate for every tenfold increase in data, it would need on the order of one billion fine-tuning tokens to match my model, which is roughly the amount of Uzbek text my model was trained on from scratch. Because it rests on two points, this is an order of magnitude, not a prediction.

The ablation suggests why the gap closes so slowly. mGPT is not held back by its tokenizer: the same tokenizer reaches 1.031 when a model is trained on a billion tokens of Uzbek. What mGPT lacks is Uzbek text.

4.4 Build or adapt? (RQ3)

The two approaches cost very different amounts. Building from scratch took a billion-token corpus, about 3.4 GPU-hours on a rented RTX 4090, and about $3.60. Fine-tuning mGPT on 1 million tokens took under 20 minutes on a free GPU and finished 0.039 bits-per-byte behind; fine-tuning on 10 million took 2.8 hours and finished 0.026 behind. By the rough extrapolation above, closing the gap completely could need about as much Uzbek text as building from scratch, at which point adapting would lose the advantage that made it attractive.

Each approach fits a different situation. Building from scratch makes sense when a large corpus is available and the best quality matters. Adapting makes sense when data or compute is scarce and good-enough quality is needed quickly. The tokenizer's role is narrower than I first thought. It did not improve prediction quality. Its benefit is efficiency: half the tokens of English-centric tokenizers, 45% fewer than mGPT's, and 21% fewer than the best multilingual one, which lowers cost and doubles usable context; and, in the ablation, comparable quality with less than half the parameters and about a third of the training computation. It also costs little: a tokenizer can be built in hours on ordinary hardware. Using a new tokenizer with a model that was pretrained with a different one is not trivial, because the model's embeddings must be re-initialized and retrained (see WECHSEL, Minixhofer et al., 2022, and FOCUS, Dobler & de Melo, 2023), and I leave testing that combination to future work.

4.5 What the model writes

Table 5 shows the first sentence my model wrote after each of four prompts. I generated one continuation per prompt with a fixed random seed and did not choose among samples, so these are not selected to look good.

Table 5. Sample output of uzbek-gpt-103m (first sentence of one continuation per prompt; fixed seeds; no selection). Created by the author.

Prompt	Continuation	English translation
Oʻzbekiston Respublikasi	qonunchiligiga muvofiq ushbu mamlakatda faoliyat koʻrsatayotgan barcha fuqarolar sogʻligʻi toʻgʻrisida toʻliq va toʻgʻri xabardor boʻlishi hamda tegishli maʼlumotlarni oʻz vaqtida taqdim etishi lozim.	Under the legislation of the Republic of Uzbekistan, all citizens active in this country must be fully and accurately informed about their health and must submit the relevant information on time.
Toshkent shahrida	ochilgan filialda faoliyat yuritib kelayotgan yoshlar bilan uchrashdi.	(He) met with young people working at a branch that opened in Tashkent.
Men oʻqishni yaxshi koʻraman, chunki	biz ham talabalar va aspirantlar uchunmiz.	I like reading, because we are also for students and graduate students.
Sunʼiy intellekt	(ANPC) — bu foydalanuvchilarga real vaqt rejimida sunʼiy intellekt algoritmlarini boshqarish imkonini beruvchi aqlli tizim.	Artificial intelligence (ANPC) is a smart system that lets users control artificial intelligence algorithms in real time.

The sentences are grammatical Uzbek and read like the web pages the model learned from: legal documents, news reports, and product descriptions. They correctly use Uzbek's stacked suffixes and vowel patterns, such as *sogʻligʻi toʻgʻrisida*. But there is no understanding behind them. The third continuation does not make sense, the fourth invents an abbreviation, and the longer continuations in the repository invent a person's name and a phone model. This is expected of a small base model trained only to predict the next word, and it is why the evaluation relies on bits-per-byte rather than on judging samples.

4.6 Design choices and limitations

Design choices. Some features of this study are deliberate. The main comparison is between two realistic approaches as a student could actually carry them out: a small model trained from scratch, and a large model fine-tuned cheaply. They differ in size and in how much Uzbek text they saw; the ablation and the fine-tuning experiment exist to separate those differences. Pretraining a 1.3-billion-parameter model on a billion Uzbek tokens would remove the difference in data, but it is exactly the expensive step that adaptation is meant to avoid. I capped evaluation at 512 tokens, although my model handles 1,024, because mGPT fails beyond 512 and a longer window would have exaggerated my model's advantage. Finally, the study covers Uzbek in Latin script only; Uzbek is also widely written in Cyrillic, which my tokenizer can encode but handles inefficiently.

Limitations. First, I evaluated only how well models predict text; I did not test them on downstream tasks or with human judges. Second, each model was trained once, so I cannot measure how much results would vary with a different random seed; four repeated runs of the mGPT fine-tuning gave results within 0.0003 bits-per-byte of one another on normalized text, but neither from-scratch model was trained twice. This matters most for the ablation, whose differences from my model (0.005 to 0.007) are small, and the bootstrap intervals cover only the choice of test documents, not the randomness of training. Third, the ablation was trained on different hardware (two T4 GPUs instead of one RTX 4090), in float16 instead of bfloat16, and with its own random order of training batches; I do not expect these to matter, but I did not test them. Fourth, the ablation matched my model's training data, not its computation, so this study cannot say which tokenizer is better at equal computation. Fifth, the test set keeps only documents that appear nowhere else in the corpus, and duplicated web pages tend to be boilerplate and syndicated news, so the results describe non-duplicated Uzbek writing specifically. Sixth, the estimate that about one billion fine-tuning tokens would close the gap rests on two points. Seventh, I did not try other vocabulary sizes, so I can say that 16,384 tokens reached comparable quality to 100,000 with fewer parameters, but not that it is the best choice. Eighth, I did not train a version without apostrophe normalization, so its separate effect is not measured, and the published normalization misses the left quote (‘), which is the single most common way people type the mark in oʻ and gʻ (about 34% of cases in a sample of 3,000 raw web documents). Treating it like the plain apostrophe would cut the number of tokens on the test documents by 2.3%, from 1.749 to 1.709 per word. Ninth, the current multilingual models in Table 3 were tested only without fine-tuning; fine-tuning them on Uzbek, as I did with mGPT, would probably improve them considerably.

5. Conclusion

In this study I built a tokenizer and a language model for Uzbek from scratch and compared them with multilingual models up to seventeen times larger, including one fine-tuned on a small part of the same Uzbek corpus, and with a copy of my model that used a multilingual tokenizer. On the three research questions:

1. **Tokenizer efficiency.** A tokenizer built for Uzbek is the most efficient of the eleven tested, at 1.749 tokens per word: about half the tokens of English-centric tokenizers and 21% fewer than the best multilingual one.

2. **Model quality.** My 103M-parameter model predicts held-out Uzbek text better than mGPT-1.3B, both zero-shot and fine-tuned (1.038 against 1.077 bits-per-byte, 95% CI for the difference [0.031, 0.049]), and much better than four current Qwen models of up to 1.7 billion parameters. This advantage comes from pretraining on a billion tokens of Uzbek: the same model trained on the same documents with mGPT's tokenizer reaches comparable quality.
3. **Build or adapt.** Adapting is fast and cheap but leaves a gap, and a rough extrapolation suggests that closing it would take about as much Uzbek text as building from scratch.

The answer to the title's question is: training data, for quality; the tokenizer, for efficiency. A tokenizer built for Uzbek did not make my model predict Uzbek better. It made the model cheaper: comparable quality with less than half the parameters, about a third of the training computation, and about half as many tokens for every text. The quality gain over multilingual models came from training on a large amount of Uzbek, which is also why current multilingual models, trained on enormous amounts of text but little of it Uzbek, predict Uzbek so poorly. A community that wants a better model for its language should expect to need text in that language; a dedicated tokenizer is the cheap way to make the resulting model smaller and faster.

This study also has a methodological lesson. My original evaluation set was built in a natural and common way, and it turned out to be the opening of my training data. A second attempt was 39% duplicated. A third, clean set was still written in my own tokenizer's spelling. And my control experiment had not been run the way I described it. Each problem made my results look better, and none was visible without an explicit check. Anyone evaluating a language model on web data should check their test set against their training data, and against how the text is really written, and should check that each experiment actually ran as described, before trusting a result. In my case the corrections removed my original main claim, that the tokenizer is what makes my model better. What remains is smaller, but I can defend it.

This work adds Uzbek to a growing movement of communities building language technology for their own languages, including the Masakhane community for African languages (Nekoto et al., 2020), Latxa for Basque (Etxaniz et al., 2024), SozKZ for Kazakh (Tukenov, 2026), and Paramanu for Indic languages (Niyogi & Bhattacharya, 2024).

Future work. The most useful next steps are evaluating the model on downstream tasks and with human judges; comparing the two tokenizers at equal training computation and with repeated runs; testing a new tokenizer with a model pretrained with a different one; and extending the approach to other Central Asian languages such as Kazakh, Kyrgyz, and Tajik. One small follow-up experiment points to another use of a dedicated tokenizer. Uzbekistan's parliament approved in 2026 an alphabet amendment that replaces sh, ch, oʻ and gʻ with ş, ç, ö and ğ. Text in the new alphabet raises my tokenizer's fertility by 38%, from 1.709 to 2.353 on the test set in normalized spelling, while multilingual tokenizers, which already know these letters from Turkish, barely change. Converting the new letters back to the old ones before tokenizing restores 1.709 exactly, with no retraining. A spelling reform, in other words, can be handled as a small preprocessing step rather than a new model.

Acknowledgments

I used Claude, an AI assistant made by Anthropic, to learn the methods, to write and debug code, and to edit this paper. It also flagged two of the four problems in Section 3.4: the possible overlap between my test and training data, and the mismatch between my ablation run and its description. I confirmed both, ran every check and the retraining, and chose to report the results even where they weakened my original claims. I built the tokenizer, trained every model, and ran every experiment myself, and I am responsible for all results and claims in this paper.

Data and code availability

The tokenizer and model weights are available on Hugging Face (huggingface.co/IslombekT/uzbek-bpe-16k; huggingface.co/IslombekT/uzbek-gpt-103m). The training code, the evaluation scripts, the ablation retraining script, the document-level bootstrap, the held-out set in both spellings, and the scores behind every table and figure are available on GitHub (github.com/TurdiyevIslombek/uzbek-gpt-from-scratch).

References

Adilova, F., Davronov, R., Kushmuratov, S., & Safarov, R. (2022). *GPTuz* [Language model]. Hugging Face. https://doi.org/10.57967/hf/0143

Ahia, O., Kumar, S., Gonen, H., Hofmann, V., Limisiewicz, T., Tsvetkov, Y., & Smith, N. A. (2024). MAGNET: Improving the multilingual fairness of language models with adaptive gradient-based tokenization. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.

Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D. R., Smith, N. A., & Tsvetkov, Y. (2023). Do all languages cost the same? Tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 9904–9923). Association for Computational Linguistics.

Bobojonova, L., Akhundjanova, A., Ostheimer, P. S., & Fellenz, S. (2025). BBPOS: BERT-based part-of-speech tagging for Uzbek. In *Proceedings of the First Workshop on Language Models for Low-Resource Languages* (pp. 287–293). Association for Computational Linguistics.

Chang, T. A., Arnett, C., Tu, Z., & Bergen, B. K. (2024). *Goldfish: Monolingual language models for 350 languages*. arXiv. https://arxiv.org/abs/2408.10441

Churchill, G., & Skiena, S. (2026). *Reducing tokenization premiums for low-resource languages*. arXiv. https://arxiv.org/abs/2601.13328

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Dobler, K., & de Melo, G. (2023). FOCUS: Effective embedding initialization for monolingual specialization of multilingual models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 13440–13454). Association for Computational Linguistics.

Etxaniz, J., Sainz, O., Perez, N., Aldabe, I., Rigau, G., Agirre, E., Ormazabal, A., Artetxe, M., & Soroa, A. (2024). Latxa: An open language model and evaluation suite for Basque. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 14952–14972). Association for Computational Linguistics.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., & Leahy, C. (2020). *The Pile: An 800GB dataset of diverse text for language modeling*. arXiv. https://arxiv.org/abs/2101.00027

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR 2022)*.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). *Scaling laws for neural language models*. arXiv. https://arxiv.org/abs/2001.08361

Kuriyozov, E., Vilares, D., & Gómez-Rodríguez, C. (2024). BERTbek: A pretrained language model for Uzbek. In *Proceedings of the 3rd Annual Meeting of the Special Interest Group on Under-resourced Languages @ LREC-COLING 2024* (pp. 33–44). ELRA and ICCL.

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR 2019)*.

Mansurov, B., & Mansurov, A. (2021). *UzBERT: Pretraining a BERT model for Uzbek*. arXiv. https://arxiv.org/abs/2108.09814

Minixhofer, B., Paischer, F., & Rekabsaz, N. (2022). WECHSEL: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 3992–4006). Association for Computational Linguistics.

Nekoto, W., Marivate, V., Matsila, T., Fasubaa, T., Fagbohungbe, T., Akinola, S. O., Muhammad, S., Kabongo Kabenamualu, S., Osei, S., Sackey, F., Niyongabo, R. A., Macharm, R., Ogayo, P., Ahia, O., Berhe, M. M., Adeyemi, M., Mokgesi-Selinga, M., Okegbemi, L., Martinus, L., … Bashir, A. (2020). Participatory research for low-resourced machine translation: A case study in African languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020* (pp. 2144–2160). Association for Computational Linguistics.

Niyogi, M., & Bhattacharya, A. (2024). *Paramanu: A family of novel efficient Indic generative foundation language models*. arXiv. https://arxiv.org/abs/2401.18034

Penedo, G., Kydlíček, H., Sabolčec, V., Messmer, B., Foroutan, N., Kargaran, A., Raffel, C., Jaggi, M., Von Werra, L., & Wolf, T. (2025). *FineWeb2: One pipeline to scale them all — Adapting pre-training data processing to every language*. arXiv. https://arxiv.org/abs/2506.20920

Petrov, A., La Malfa, E., Torr, P. H. S., & Bibi, A. (2023). Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Qin, L., Chen, Q., Zhou, Y., Chen, Z., Li, Y., Liao, L., Li, M., Che, W., & Yu, P. S. (2025). A survey of multilingual large language models. *Patterns, 6*(1), Article 101118. https://doi.org/10.1016/j.patter.2024.101118

Qwen Team. (2024). *Qwen2.5 technical report*. arXiv. https://arxiv.org/abs/2412.15115

Qwen Team. (2025). *Qwen3 technical report*. arXiv. https://arxiv.org/abs/2505.09388

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI technical report.

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1715–1725). Association for Computational Linguistics.

Shazeer, N. (2020). *GLU variants improve transformer*. arXiv. https://arxiv.org/abs/2002.05202

Shliazhko, O., Fenogenova, A., Tikhonova, M., Kozlova, A., Mikhailov, V., & Shavrina, T. (2024). mGPT: Few-shot learners go multilingual. *Transactions of the Association for Computational Linguistics, 12*, 58–79. https://doi.org/10.1162/tacl_a_00633

Solidjonov, D., & Najmiddinov, M. (2026). Investigating linguistic errors in large language model generation of Uzbek text. *Cogent Arts & Humanities, 13*(1), Article 2600519. https://doi.org/10.1080/23311983.2025.2600519

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing, 568*, 127063.

Tukenov, S. (2026). *SozKZ: Training efficient small language models for Kazakh from scratch*. arXiv. https://arxiv.org/abs/2603.20854

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.

Zhang, B., & Sennrich, R. (2019). Root mean square layer normalization. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*.